Prometheus - ESP32-C3 Gateway

Compilacao e gravacao

Use o pacote ESP32 3.3.12 e GFX Library for Arduino 1.6.9, disponiveis no ambiente de validacao. Selecione **ESP32C3 Dev Module** e **Minimal SPIFFS (1.9MB APP with OTA/128KB SPIFFS)**, ou um layout OTA maior. Esse esquema precisa de pelo menos 4 MB de flash; confira a placa fisica. O firmware com Prometheus excede a particao padrao de 1,25 MiB.

FQBN correspondente: `esp32:esp32:esp32c3:PartitionScheme=min_spiffs`. Ao mudar a tabela de particoes, grave por USB; um upload OTA somente do aplicativo nao atualiza essa tabela. O layout Minimal SPIFFS mantem a regiao NVS no mesmo endereco do layout padrao, mas nao use a opcao de apagar toda a flash se quiser preservar as preferencias.

Configuracao web e portas

1. Abra `http://IP_DO_ESP32/prometheus`, pelo painel ou pela pagina Wi-Fi. A interface web permanece na **porta 80**, inclusive no AP `ESP32_C3_CONFIG` (`192.168.4.1`).
2. Informe o endereco do servidor Prometheus: IPv4 ou DNS, sem protocolo, caminho ou porta; por exemplo, `192.168.1.10` ou `prometheus.local`.
3. Escolha a **porta HTTP de metricas do ESP32**, de 1 a 65535, padrao **9120**. Nao e a porta da interface do servidor Prometheus.
4. Escolha o intervalo sugerido de coleta, de 5 a 3600 segundos, padrao **15**.
5. Salve. As preferencias sao persistidas na NVS. Uma mudanca de porta e aplicada cerca de um segundo depois, sem reboot ou redirecionamento do painel.
6. Baixe `/prometheus.yml`, adicione o job ao servidor e recarregue sua configuracao. Reserve o IP do ESP32 no DHCP.

O modelo e **pull**, sem Pushgateway: o Prometheus consulta `http://IP_DO_ESP32:PORTA/metrics`. O endereco do servidor e uma referencia persistente; nao configura envio ativo, autorizacao ou o servidor remoto. O intervalo so e efetivo quando o YAML e aplicado no Prometheus.

Servico	Porta
Painel, Wi-Fi, configuracao Prometheus e download YAML	**80**, fixa
Endpoint `/metrics`	Configurada e salva; padrao **9120**

Quando a porta configurada e diferente de 80, `/metrics` retorna 404 na porta 80 e o servidor de metricas nao oferece as paginas do painel. Se configurar 80, apenas `/metrics` compartilha o servidor web; nao e criado um segundo listener nessa porta. A porta anterior deixa de atender apos a aplicacao da mudanca. As mesmas regras se aplicam em STA e AP e depois de reiniciar.

Os endpoints nao possuem autenticacao. Use em rede confiavel; nao exponha o ESP32 diretamente a Internet. O endereco salvo nao restringe os clientes. Consultas de clima e outras operacoes sincronas do loop podem atrasar o HTTP.

Configuracao do servidor Prometheus

Exemplo, substituindo IP e porta pelos valores reais do ESP32:

```
scrape_configs:
  - job_name: 'esp32_c3'
    scrape_interval: 15s
    metrics_path: /metrics
    static_configs:
      - targets: ['192.168.1.42:9120']
```

Se ja existe `scrape_configs:`, acrescente apenas o job, sem duplicar a secao. Use jobs com nomes unicos ou agrupe os dispositivos nos targets. O download usa o IP STA quando conectado ou o IP AP quando offline: gere novamente o YAML se o IP do alvo mudar. A porta do servidor Prometheus, normalmente 9090, nao precisa ser igual a porta de metricas do ESP32.

Persistencia e tratamento de erros

O namespace Preferences `telemetry` guarda:

- `settings`: registro versionado com endereco, porta e intervalo.
- `boots`: registro versionado com inicializacoes e resets por watchdog.

As configuracoes sao carregadas antes da inicializacao dos servidores. Entrada invalida retorna HTTP **400**, sem alterar a configuracao. Falha ao salvar retorna **503**, com aviso na pagina e na Serial, preservando o estado ativo em RAM. Registros invalidos ou NVS indisponivel sao informados na Serial. Sem configuracao valida, a porta padrao e 9120 e o intervalo e 15 segundos.

Boot/watchdog sao gravados uma vez por inicializacao, nunca a cada coleta. Se o registro desses contadores estiver invalido ou nao puder ser gravado, as respectivas series sao omitidas, com aviso na pagina e na Serial. Os contadores saturam no limite de 32 bits.

`reset_wifi` e a limpeza por BOOT nao apagam `telemetry`; a limpeza por BOOT continua apagando somente `wifi` e `ntp_cfg`. Uma limpeza completa da NVS por ferramentas externas remove tambem as configuracoes e os contadores Prometheus.

Metricas publicadas

Formato Prometheus **0.0.4**, com `HELP`, `TYPE` e labels escapados. Todas as series possuem `device`, o MAC STA do dispositivo, estavel tambem no AP. Informacoes textuais usam gauges com valor 1 e labels.

Grupo	Metricas	Interpretacao
Identificacao	`esp_info`	Valor 1, labels `chip` e `model`.
Identificacao	`esp_firmware_version`	Valor 1; label `version` usa a mesma versao automatica do comando `version`; `build` informa data/hora da compilacao.

Grupo	Metricas	Interpretacao
Identificacao	`esp_ip_address`	Valor 1; labels `ip` e `interface` (`sta` ou `ap`), por interface ativa.
Identificacao	`esp_mac_address`	Valor 1; label `mac` da interface STA.
Disponibilidade	`esp_status_online`	1 se STA conectado, 0 caso contrario; nao testa Internet.
Disponibilidade	`esp_uptime_seconds`	Timer monotono de 64 bits, em segundos desde o boot.
Disponibilidade	`esp_last_restart_reason`	Valor 1; codigo `esp_reset_reason_t` no label `reason`.
Disponibilidade	`esp_boot_count`	Inicializacoes acumuladas na NVS.
Wi-Fi	`esp_wifi_rssi_dbm`, `esp_wifi_channel`	RSSI e canal; omitidos sem conexao STA.
Wi-Fi	`esp_wifi_disconnect_count`	Eventos de desconexao desde boot, inclusive desconexoes deliberadas e tentativas.
Wi-Fi	`esp_wifi_tx_power`	Limite configurado no driver em dBm; nao mede consumo. Omitido sem STA ou se a leitura falhar.
Processador	`esp_cpu_frequency_mhz`	Frequencia atual.
Processador	`esp_cpu_usage_percent`	Uso agregado dos nucleos, de 0 a 100%; ver limitacoes.
Processador	`esp_cpu_temperature_celsius`	Temperatura interna do chip em graus Celsius, via `temperatureRead()`; omitida se a leitura nao for finita. Nao e temperatura ambiente.
Clima	`esp_weather_data_age_seconds`	Segundos desde a ultima atualizacao valida do cache Open-Meteo, com timer monotono de 64 bits. Omitida antes da primeira sincronizacao valida.
Processador	`esp_task_count`	Quantidade atual de tarefas FreeRTOS.
Memoria	`esp_heap_free_bytes`, `esp_heap_total_bytes`, `esp_heap_minimum_free_bytes`	Heap livre, total e minimo livre desde boot.
Memoria	`esp_psram_free_bytes`, `esp_psram_total_bytes`	PSRAM livre e total; zero quando nao ha PSRAM disponivel.

Grupo	Metricas	Interpretacao
GPIO	`gpio_digital_input`	Nivel instantaneo dos botoes GPIO8, GPIO9 e GPIO10, identificados pelo label `pin`: 0 pressionado, 1 solto.
GPIO	`gpio_digital_output`	Nivel logico instantaneo do LED GPIO11; label `pin`.
Sistema	`esp_reboot_count`	Inicializacoes acumuladas menos a primeira, incluindo ciclos de energia e retorno de deep sleep.
Sistema	`esp_watchdog_resets`	Total persistido de resets por watchdog de tarefa, interrupcao e geral.
Sistema	`esp_free_rtos_tasks`	Mesma quantidade de tarefas de `esp_task_count`.
Sistema	`esp_flash_usage_percent`	Tamanho do firmware / capacidade da particao em execucao, em porcentagem. Nao e uso de todo o chip flash.
Aplicacao	`application_requests_total`	Counter de requisicoes HTTP concluidas nos dois servidores.
UDP	`esp_udp_listener_up`	Gauge: 1 quando a abertura da porta UDP 4210 teve sucesso; 0 quando falhou. Nao verifica conectividade ou entrega ao cliente.
UDP	`esp_udp_packets_received_total`	Counter de datagramas detectados por `parsePacket()`, incluindo comandos vazios ou invalidos.
UDP	`esp_udp_replies_sent_total`	Counter de datagramas de resposta enviados com sucesso pela API UDP local. Nao confirma recebimento pelo cliente.
UDP	`esp_udp_send_errors_total`	Counter de falhas ao iniciar, escrever ou finalizar um datagrama de resposta.
Aplicacao	`application_errors_total`	Counter de respostas 4xx/5xx ou sem codigo de resposta.
Aplicacao	`application_response_time_ms`	Gauge do tempo de tratamento e envio da ultima requisicao HTTP concluida.

Limitacoes

- CPU requer estatisticas FreeRTOS de runtime e duas amostras validas, separadas por cinco segundos. A serie e omitida no aquecimento, sem suporte do SDK, quando faltam tarefas idle ou quando a amostra

e invalida. O calculo usa runtime em microssegundos do ESP-IDF, somando idle e normalizando pelos nucleos.

- Os contadores HTTP e de desconexao sao em RAM e reiniciam no boot. O scrape atual so entra nas estatisticas depois que a resposta termina. Tempo de resposta nao e media, percentil, nem latencia total do cliente.
- Os contadores UDP tambem sao em RAM e reiniciam no boot. Respostas longas sao divididas em datagramas de ate 1200 bytes, contados individualmente. Sem destinatario UDP conhecido, mensagens locais nao tentam envio e nao incrementam erros. A resposta de limpeza do Wi-Fi usa os mesmos contadores.
- A idade do clima zera somente em uma atualizacao valida e continua crescendo quando as consultas falham ou o Wi-Fi cai. Nao mede a idade da observacao na origem, mas o tempo desde que o ESP32 aceitou a resposta da API. NTP e a virada do contador `millis()` nao alteram esse calculo. O cache e em RAM: depois de reiniciar, a serie fica ausente ate uma nova sincronizacao valida.
- GPIO e uma leitura instantanea, nao um contador de pulsos. A coleta nao altera os modos dos pinos nem reconfigura barramentos ou ADC.
- O dispositivo desligado nao publica `esp_status_online=0`: use a metrica `up` do proprio Prometheus para detectar indisponibilidade.

Metricas indisponiveis

Conforme o escopo escolhido, somente fontes reais integradas sao publicadas. Estas series **nao sao emitidas com valores ficticios ou zeros simulados**:

Grupo	Metricas	O que falta
Energia	`esp_voltage_v`, `esp_current_ma`, `esp_power_w`	Medidor eletrico e leituras calibradas.
Bateria	`esp_battery_percent`, `esp_battery_voltage_v`	Medicao calibrada e curva da bateria.
Ambiente	`sensor_temperature_celsius`, `sensor_humidity_percent`, `sensor_pressure_hpa`, `sensor_air_quality`	Sensores locais integrados; a API de clima nao e sensor local.
Reles	`relay_state`, `relay_activation_count`, `relay_runtime_seconds`	Mapeamento e controle de reles com instrumentacao de transicoes.
ADC	`gpio_analog_value`	Pinos, escalas e leituras ADC sem conflito com o hardware existente.
MQTT	`mqtt_connected`, `mqtt_messages_sent`, `mqtt_messages_received`, `mqtt_publish_errors`	Cliente MQTT e instrumentacao de conexao, publicacao e recepcao.

Verificacao na placa

1. Acesse painel e `/prometheus` na porta 80 em STA e AP.

2. Salve host, porta 9121 e intervalo 20. Confira o YAML com `:9121` e `20s`.
3. Apos um segundo, confira HTTP 200 em `http://IP:9121/metrics`, o content-type Prometheus e HTTP 404 em `http://IP/metrics`. O painel deve continuar na 80.
4. Reinicie: confira host, porta e intervalo restaurados e aumento dos contadores de boot/reboot; watchdog nao deve aumentar em reboot normal.
5. Troque para porta 80 e depois de volta para 9121: `/metrics` deve responder somente na porta ativa, mantendo painel e formulario na 80.
6. Envie host com protocolo, porta 0/65536 e intervalo 4/3601: cada tentativa deve retornar 400 sem alterar a configuracao salva.
7. Observe RSSI/canal apenas em STA, CPU apos aquecimento, GPIO com botoes/LED e contadores HTTP apos uma requisicao valida e uma rota inexistente.
8. Aplique o YAML no servidor e confira `up{job="esp32_c3"} == 1` e as series acima. Desligue o ESP e confirme `up == 0`.
9. Confira `esp_cpu_temperature_celsius` com o comando `temp`. Antes da primeira consulta valida do clima, `esp_weather_data_age_seconds` deve estar ausente; depois, deve crescer entre coletas e voltar a proximo de zero apos `clima_sync` bem-sucedido. Uma consulta com falha nao deve zerar a idade.

A compilacao nao substitui essas verificacoes de rede, NVS e hardware na placa.